

MediTrust

Render Deployment Notes

Single-service deploy: Express serves the API + the React build | 17 Sep 2026

These notes document how MediTrust was made deployable on Render: the architecture, the exact code that lets the server serve the frontend, and every deployment error we hit with its root cause and fix. Keep this with the project for the next deploy.

1. Deployment architecture (single service)

Instead of two services (API + static site), the whole platform runs as ONE Render web service. Express answers `/api/*` itself and serves the compiled React app for everything else:

```
Browser  --->  https://meditrust.onrender.com/
              |
            /api/*  ----+----  Express routers (auth, products, cart, ...)
            /health ----+----  Render health check
            /*      ----+----  client/dist/index.html (React SPA)
              |
              +----  MongoDB Atlas, Gemini API, Cloudinary
```

Why one service: no CORS to configure between frontend and backend, one URL, free-tier friendly, and the frontend build always matches the backend it talks to.

Render settings used

Setting	Value
Runtime	Node (>= 20 via engines field)
Build command	npm run build (= root install + client install + vite build)
Start command	npm start (= node server/server.js)
Health check	GET /api/health
Secrets (dashboard)	MONGO_URI, JWT_SECRET (auto), GEMINI_API_KEY, CLOUDINARY_*
Optional	FRONTEND_URL = public URL, used for CORS allowlist

2. The code that serves the frontend

2.1 server/server.js - static + SPA fallback

Placed AFTER all `/api/*` routes and BEFORE the error handler, so API routes always win and only non-API paths fall through to the app:

```

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const clientDist = path.join(__dirname, '..', 'client', 'dist');

if (fs.existsSync(clientDist)) {
  app.use(express.static(clientDist));
  // SPA fallback: everything that is NOT /api/* serves index.html
  app.get(/^(?!\/api).*/, (req, res) => {
    res.sendFile(path.join(clientDist, 'index.html'));
  });
}

```

- express.static serves real files first (JS/CSS/images with correct caching headers).
- The regex `/^(?!\/api).*/` matches any path NOT starting with `/api`, so `/shop` or `/orders/123` return `index.html` and React Router takes over inside the browser (this is what makes page refresh work on deep links).
- Unknown `/api/*` paths never reach the fallback, so API 404s stay API 404s.
- The `existsSync` guard means local backend dev still boots when `client/dist` was never built.

2.2 client/src/services/api.js - same-origin API base

Vite bakes env vars into the bundle at BUILD time, when the Render URL is not yet known. So production builds ignore the env var and call the same origin:

```

const api = axios.create({
  baseURL: import.meta.env.PROD
    ? '/api' // Render: same service
    : `${import.meta.env.VITE_API_BASE_URL ||
      'http://localhost:8080'}/api`, // local vite dev
});

```

- `import.meta.env.PROD` is replaced by Vite at build time (true only for ``vite build``), so the production bundle hard-codes `baseURL: '/api'` - verified inside `dist/assets/*.js`.
- Local ``vite` dev` (`PROD=false`) still uses `client/.env` pointing at `localhost:8080`.

2.3 package.json scripts + render.yaml

```

"build": "npm install && npm --prefix client install
        && npm --prefix client run build",
"start": "node server/server.js", // plain node, never --watch
"engines": { "node": ">=20" },

```

`render.yaml` (Blueprint) pins `buildCommand/startCommand/healthCheckPath` and declares secrets, so a fresh service can be recreated from code. `JWT_SECRET` uses `generateValue:true` (Render creates a strong random secret).

2.4 CORS + health check

```

const allowedOrigins = (process.env.FRONTEND_URL || '')
  .split(',')
  .map(o => o.trim())
  .filter(Boolean);
app.use(cors(allowedOrigins.length ? { origin: allowedOrigins } : {}));

app.get('/api/health', (req, res) =>
  res.status(200).json({ status: 'ok', time: new Date().toISOString() }));

```

- Empty `FRONTEND_URL` = open CORS in dev; set it to the public URL in production to lock the API to your domain.
- `/api/health` is what Render polls to decide the deploy is LIVE.

3. Errors we hit, and how each was solved

E1 - Build fails: ``sh: 1: vite: not found``

Log evidence: client install reported only ~52 packages, then `vite build` failed with `vite: not found`.

- Root cause: Render sets NODE_ENV=production, and `npm install` under production SKIPS devDependencies. vite, @vitejs/plugin-react, tailwindcss and @tailwindcss/vite were all in client devDependencies, so the Vite binary was never installed.

- Fix: moved those four packages into client `dependencies` (12 -> 13 entries, zero devDeps). Rule of thumb: anything the HOST needs to build or run your app must be a real dependency, not a devDependency.

E2 - Deploy crashes: `concurrently: not found`, exit 127

Log evidence: `==> Running 'npm run dev' ... `sh: 1: concurrently: not found` ... `Exited with status 127`.

- Root cause: the service's Start Command was `npm run dev` (a local-dev command: file-watch + Vite dev server + concurrently). concurrently is a devDependency, pruned in production installs, so the shell could not find it. Exit 127 = command not found.

- Fix: Start Command set to `npm start` (plain `node server/server.js` serving API + dist). The concurrently path is never touched in production. Lesson: `dev` is for laptops, `start` is for servers.

E3 - Stale code deployed (old build script ran)

Log evidence: build ran `npm --prefix client install && ...` with NO leading root `npm install`, matching an older commit (a8113a1) even though the fix was already written locally.

- Root cause: the fix had not been committed/pushed - Render builds the pushed commit, not your working folder.
- Fix: commit + push, then redeploy. Always compare the commit hash in the Render log with your latest local commit when a fix 'does not take effect'.

E4 - `Permission denied`, exit 126 (start command without runtime)

Log evidence: `==> Running 'server/server.js' ... `bash: line 1: server/server.js: Permission denied` ... `Exited with status 126`.

- Root cause: the Start Command field contained only the file path, without the `node` runtime in front. Bash then tried to EXECUTE the .js file as a program. Exit 126 = 'found it, but it is not executable' (vs 127 = 'not found').

- Fix: Start Command exactly `npm start`, which resolves `node server/server.js` through package.json - no path or prefix to get wrong.

E5 (near-miss) - trailing comma broke client/package.json

While moving dependencies, an edit left `},` before the final `}` - invalid JSON that would have failed the next deploy at parse time. Caught locally by running JSON.parse over the file before pushing.

- Lesson: after hand-editing any package.json, validate it (quick JSON.parse) before committing. One comma can block a whole deploy.

4. Pre-deploy checklist (next time)

#	Check	How
1	Build tools are in dependencies, not devDependencies	npm pkg get dependencies (client)
2	package.json files parse as JSON	JSON.parse read of each file
3	Start command runs the built app, not dev tooling	render.yaml startCommand = npm start
4	Pushed commit == commit in Render log	git log vs 'Checking out commit ...' line
5	Secrets set in dashboard, never committed	Render env tab; .env stays gitignored
6	MongoDB Atlas allows Render IPs	Atlas Network Access: 0.0.0.0/0 (dynamic IPs)
7	First deploy: clear build cache	Manual Deploy -> Clear build cache & deploy

5. Key takeaways

- Exit 127 = command not found (missing binary, often pruned devDeps). Exit 126 = found but not executable (missing `node` prefix). Reading the exact `Running ...` line tells you which.
- `NODE_ENV=production` changes install behaviour: devDependencies are skipped. Build on the host => build tools must be real dependencies.
- Vite bakes env vars at BUILD time: for same-service deploys, prefer `import.meta.env.PROD` branching over build-time URLs you cannot know yet.
- One Render web service can host API + SPA: static middleware first, then a non-/api SPA fallback, health check for the platform.